

Modern 2D Browser Game Architecture Toolkit

Final Engineering Handbook, Agent Contract, and Operational Tooling Guide

Version 2.0 — 18 August 2026

***Governing principle:** The game’s requirements determine the architecture. Begin with the simplest implementation that satisfies required behavior, visual quality, accessibility, reliability, maintainability, and target-device constraints. Introduce specialized rendering, physics, memory management, animation, concurrency, generated-asset infrastructure, or deployment systems only when explicit requirements, existing project architecture, profiling, scale, browser constraints, or measured failure modes justify them.*

This document is not a universal game engine specification. It is a **decision and enforcement system** for humans and AI coding agents building 2D browser games. It becomes operational through the companion project contract, preflight compiler, linter, asset-manifest validator, validation-checklist generator, optional browser smoke harness, CI recipe, templates, and self-test shipped with this handbook.

Contents

- 0. What This Toolkit Is For

PART I — AUTHORITY, ARCHITECTURE, AND PROJECT CONTRACTS

- 1. Normative and Evidence Terminology
- 2. The Architecture Project Contract
- 3. Architecture Decision Framework
- 4. Existing-Project Preservation
- 5. Reference Architecture Profiles

PART II — BROWSER EXECUTION AND GAMEPLAY ARCHITECTURE

- 6. Timing Models
- 7. Browser Lifecycle
- 8. High-DPI and Coordinate Architecture
- 9. State Ownership and Scene Lifecycle
- 10. Input Architecture and Analog Deadzones

PART III — ASSETS, RENDERING, ANIMATION, AND PHYSICS

- 11. Asset Authority and the Three-Stage Verified Lifecycle
- 12. Rendering Path Selection
- 13. WebGL2 Resource and Failure Model
- 14. Lighting, Normal Maps, and Postprocessing
- 15. Animation and Camera
- 16. Collision and Physics Escalation

PART IV — PERFORMANCE, CONCURRENCY, AUDIO, STORAGE, AND SECURITY

- 17. Memory and Performance Methodology
- 18. Workers, OffscreenCanvas, and Shared Memory
- 19. Web Audio
- 20. Persistence and Schema Evolution
- 21. Accessibility and Browser UI
- 22. Security and Deployment Boundaries

PART V — TESTING, EVIDENCE, AND THE TOOLKIT

- 23. Testing and Evidence Ladder
- 24. Toolkit Components
- 25. Architecture Linter Rule Set

- 26. Architecture Decision Records and Agent Handoffs
- 27. AI Coding Agent Operating Contract

PART VI — TEN CONCRETE BUILD EXAMPLES

- 28. Example One — Accessible DOM Card Battler
- 29. Example Two — High-DPI Canvas Arcade Dodger
- 30. Example Three — Fixed-Step Platformer
- 31. Example Four — Generated-Art Top-Down Adventure
- 32. Example Five — WebGL Particle Arena
- 33. Example Six — Accessible Puzzle Museum
- 34. Example Seven — IK Creature Sandbox
- 35. Example Eight — Swarm Collision Lab
- 36. Example Nine — Offline Strategy Simulation
- 37. Example Ten — Cross-Origin-Isolated Field Simulator

PART VII — OPERATING THE TOOLKIT IN REAL WORK

- 38. Recommended Project Lifecycle
- 39. Failure Interpretation
- 40. What This Toolkit Does Not Do

PART VIII — CURRENT PLATFORM SOURCE MAP

- 41. Glossary
- **42. Final Operational Checklist**

0. What This Toolkit Is For

Use this toolkit when you want an AI agent or human developer to build, continue, repair, or expand a browser game **without allowing architectural fashion to substitute for evidence**.

The toolkit solves five recurring problems:

1. **Architecture drift.** A small game acquires WebGL, workers, physics engines, pooling, or custom scene systems merely because they sound serious.
2. **Agent overreach.** An AI replaces a functioning Phaser/Pixi/Canvas implementation while implementing an unrelated feature.
3. **Generated-asset hallucination.** Runtime code is written against filenames, atlas frames, pivots, or dimensions that do not exist yet.
4. **False completion.** Source code looks plausible, but the real browser path, lifecycle, asset pipeline, or performance claim was never tested.
5. **Lost decisions.** The reasoning that activated or rejected a capability disappears, so another agent reopens settled architecture questions later.

The solution is deliberately boring: **declare the project contract, compile the relevant obligations, lint high-confidence violations, validate generated facts, execute the actual project checks, and record meaningful architecture decisions.**

0.1 The five-minute workflow

For a new or existing game repository:

```
TOOLKIT=/path/to/modern_2d_browser_game_architecture_toolkit

# One-time bootstrap. Does not overwrite an existing contract.
python "$TOOLKIT/tools/bootstrap_project.py" . --name "My Browser Game"

# Edit architecture.project.json so it describes the real project.

# Run the contract-driven checks.
python "$TOOLKIT/tools/run_project_checks.py" . --strict-lint
```

For generated assets, add physical asset verification:

```
python "$TOOLKIT/tools/run_project_checks.py" . \
  --strict-lint \
  --verify-asset-files \
  --verify-asset-hashes
```

When a local test server is running and Playwright is installed:

```
python "$TOOLKIT/tools/run_project_checks.py" . --browser
```

The generated `.architecture-checks/` folder contains the activated architecture report, project-specific validation checklist, machine-readable lint output, optional manifest validation, and a summary record.

PART I — AUTHORITY, ARCHITECTURE, AND PROJECT CONTRACTS

1. Normative and Evidence Terminology

Technical handbooks become dangerous when ordinary advice quietly turns into law. This one uses explicit strength labels.

- **REQUIRED** — an invariant inside the stated scope.
- **RECOMMENDED** — a strong default with legitimate exceptions.
- **CONDITIONAL** — activated only by an identified project requirement or evidence state.
- **OPTIONAL** — available capability, not required for correctness.
- **EXAMPLE** — illustrative only; never an architectural threshold.
- **MEASURE** — decide from profiling, observation, or benchmark evidence.
- **AVOID** — usually a bad tradeoff, but not logically forbidden in every context.
- **FORBIDDEN** — unacceptable behavior, such as fabricated generated-asset facts, exposed credentials, or false validation claims.

Evidence also needs names:

- **SOURCE-VERIFIED** — checked against an identified current specification or official platform document.
- **SYNTAX-VERIFIED** — parsed or checked by the relevant language tool.
- **TEST-EXECUTED** — a unit/integration/fixture test actually ran.
- **BROWSER-EXECUTED** — the named path executed in a real browser runtime.
- **VISUALLY-VERIFIED** — the visible result was inspected against a visual requirement.
- **PERFORMANCE-MEASURED** — the claimed workload was profiled under a named scenario/environment.
- **IMPLEMENTED-UNVERIFIED** — code exists but decisive behavioral proof does not.

A stronger word requires stronger evidence. A Node syntax check cannot become “browser tested” through phrasing.

2. The Architecture Project Contract

The operational center of this toolkit is `architecture.project.json`. It is a small machine-readable statement of which capabilities the project actually uses.

A typical contract:

```
{  
  "schemaVersion": 1,  
  "projectName": "My Browser Game",
```

```

"renderPath": "canvas2d",
"timingModel": "variable",
"existingFramework": null,
"targetBrowsers": ["current Chromium", "current Firefox", "current Safari"],
"capabilities": {
  "generatedAssets": false,
  "assetManifest": null,
  "webAudio": false,
  "gamepad": false,
  "workers": false,
  "sharedArrayBuffer": false,
  "deploymentAutomation": false,
  "postprocessing": false,
  "normalMapping": false,
  "inverseKinematics": false,
  "spatialBroadphase": false,
  "persistence": "none"
},
"paths": {
  "frontend": ["src", "public"],
  "ignore": ["node_modules", "dist", "build", ".git", "coverage"],
  "allowedAssetLiteralFiles": ["src/assets", "src/assetLoader.js", "src/assetLoader.ts"]
},
"validation": {
  "commands": ["npm test", "npm run build"],
  "browserUrl": "http://127.0.0.1:5173",
  "readyExpression": "() => document.readyState === 'complete'"
},
"lint": {
  "ignoreCodes": [],
  "ignorePaths": []
}
}

```

2.1 Why the contract exists

The contract does not describe every implementation detail. It describes **activation state**. If `sharedArrayBuffer` is `false`, an agent must not introduce shared memory as an optimization hobby. If generated assets are true, the project must name the authoritative asset manifest. If the timing model is event-driven, the agent should not create a permanent `requestAnimationFrame` loop merely to fit a template.

The contract also makes lint findings more meaningful. A direct asset path is not inherently wrong; it becomes suspicious when the project has declared a verified generated-asset pipeline. A `SharedArrayBuffer` reference is not inherently wrong; it is suspicious when the project never activated the security/deployment consequences.

2.2 Contract changes are architectural changes

When a feature genuinely activates a capability, update the contract in the same work unit. If that change has meaningful cost or migration impact, record an Architecture Decision Record (ADR). The contract is current state; the ADR records why that state changed.

2.3 Existing projects

For an existing repository, the contract must describe reality rather than forcing the repository to match the toolkit. If a project already uses Phaser, set `renderPath` to `existing` and name the framework. Do not translate the whole project to raw Canvas simply because Canvas has a chapter in this handbook.

3. Architecture Decision Framework

Use the following order. It avoids expensive downstream decisions being made before simpler upstream questions are answered.

1. **Requirement intake.** What player-visible behavior is actually required?
2. **Existing state.** What renderer, lifecycle, asset, input, persistence, and test systems already work?
3. **Execution model.** Is the experience event-driven, variable-delta, fixed-step, or fixed-step with interpolation?
4. **Render path.** DOM/CSS, Canvas 2D, WebGL2, existing renderer/framework, or hybrid?
5. **State ownership.** What is authoritative, derived, transient, persistent, or asynchronous?
6. **Asset authority.** Authored/static files, build outputs, generated assets, or a mixed pipeline?
7. **Input scope.** Direct input or a logical action layer? Keyboard, pointer, touch, gamepad?
8. **Collision/physics.** What is the simplest shape and broad/narrow-phase model that satisfies gameplay?
9. **Optional systems.** Audio, persistence, workers, postprocessing, IK, generated normals, deployment.
10. **Validation.** What proves the required path works?

Every meaningful escalation should be expressible as:

Problem → Evidence → Simplest credible intervention → Validation → Rollback path.

If the evidence field is empty, the escalation is speculative.

4. Existing-Project Preservation

REQUIRED: NO UNSOLICITED ENGINE REWRITE.

Before modifying an existing project, inspect the smallest set of surfaces that establishes current ownership:

- package/build manifest;
- application/game entry point;
- renderer/framework initialization;
- scene/game-state owner;
- asset loading/catalog/manifest surfaces;
- input routing;
- collision/physics owner;
- audio owner;
- persistence/schema owner;
- relevant tests and build commands.

Record known-good player journeys before editing foundations. Examples:

- launches to title screen;
- keyboard movement works;
- level transition unloads old scene listeners;
- gamepad reconnect is tolerated;
- save from schema 2 migrates to schema 3;
- generated asset ID `player.idle` resolves through the current manifest;
- pause suspends gameplay but not the settings menu.

A local refactor is a failure if it breaks protected behavior outside the requested scope.

4.1 When a migration is justified

A foundational replacement is justified when at least one is true:

- the user explicitly requests it;
- a required capability cannot be implemented safely in the current architecture;
- a dependency is incompatible with the target platform and migration is authorized;
- profiling demonstrates the current architecture cannot meet a declared performance requirement and narrower interventions fail;
- the existing subsystem is structurally broken and a bounded replacement is lower-risk than repair.

Write an ADR before a consequential migration. The included templates/ADR.md supplies the structure.

5. Reference Architecture Profiles

These are starting shapes, not maturity levels.

Profile A — DOM / interaction-heavy

For card games, board games, text adventures, management screens, turn-based puzzles, and accessibility-heavy interactions.

- semantic DOM and CSS;
- event-driven state transitions;
- no continuous loop unless a specific effect needs one;
- browser-native controls where practical;
- optional persistence.

Do **not** add a Canvas renderer merely to make it “game-like.”

Profile B — Minimal Canvas

For arcade prototypes, small action games, drawing-heavy puzzles, visual toys.

- one Canvas 2D surface;
- variable-delta loop where stable physics integration is unnecessary;
- explicit DPR/resize/pointer mapping;
- simple collision primitives.

Profile C — Stable action simulation

For platformers, collision-heavy action, physics puzzles.

- fixed simulation tick;
- bounded backlog policy;
- optional previous/current interpolation for presentation;
- action-based input if multiple devices/rebinding matter.

Profile D — Asset-heavy/generated game

For content-rich sprite games and generative art workflows.

- three-stage asset lifecycle;
- manifest as runtime authority;
- asset build before code integration;
- explicit missing/stale/corrupt behavior;
- persistence/versioning where required.

Profile E — Advanced GPU 2D

For custom shader effects, GPU particles, normal-mapped sprites, or a measured Canvas bottleneck.

- WebGL2 or an existing GPU renderer;
- explicit GPU resource lifecycle;
- shader failure handling;
- context-loss policy;
- measured postprocessing/overdraw costs.

Being Profile E does not imply workers, IK, cloud deployment, custom physics, or object pooling.

PART II — BROWSER EXECUTION AND GAMEPLAY ARCHITECTURE

6. Timing Models

Choose the timing model intentionally.

6.1 Event-driven / turn-based

Use when nothing needs continuous simulation. State changes occur in response to input, timers, network events, or explicit transitions. This is often the best model for card games, turn-based systems, menus, and many puzzle games.

Advantages:

- little idle CPU use;
- simpler state reasoning;
- no delta-time integration;
- accessibility and test automation often easier.

Do not manufacture a permanent animation loop for a project whose world does not continuously evolve.

6.2 Variable-delta loop

Use for lightweight continuous movement, UI-like animation, visual toys, and games whose collision/integration remains stable under bounded variable dt.

```
class VariableLoop {
  constructor(update, render, maxDeltaSeconds = 0.1) {
    if (typeof update !== 'function' || typeof render !== 'function') {
      throw new TypeError('update and render must be functions');
    }
    if (!Number.isFinite(maxDeltaSeconds) || maxDeltaSeconds <= 0) {
      throw new RangeError('maxDeltaSeconds');
    }
  }
}
```

```

    }
    this.update = update;
    this.render = render;
    this.maxDeltaSeconds = maxDeltaSeconds;
    this.running = false;
    this.last = 0;
    this.raf = null;
    this.frame = this.frame.bind(this);
  }
  start() {
    if (this.running) return;
    this.running = true;
    this.last = performance.now();
    this.raf = requestAnimationFrame(this.frame);
  }
  stop() {
    this.running = false;
    if (this.raf !== null) cancelAnimationFrame(this.raf);
    this.raf = null;
  }
  frame(now) {
    if (!this.running) return;
    const dt = Math.min(Math.max((now - this.last) / 1000, 0), this.maxDeltaSeconds);
    this.last = now;
    this.update(dt);
    this.render();
    this.raf = requestAnimationFrame(this.frame);
  }
}

```

The delta clamp is a product policy: it prevents a huge single update after a stall but intentionally allows simulation time to fall behind wall time.

6.3 Fixed timestep

Use when stable integration intervals materially improve gameplay correctness.

```

class FixedLoop {
  constructor(update, render, { hz = 60, maxFrame = 0.25, maxSteps = 8 } = {}) {
    if (!Number.isFinite(hz) || hz <= 0) throw new RangeError('hz');
    this.update = update;
    this.render = render;
    this.step = 1 / hz;
    this.maxFrame = maxFrame;
    this.maxSteps = maxSteps;
    this.accumulator = 0;
    this.dropped = 0;
  }
}

```

```

}
advance(frameSeconds) {
  this.accumulator += Math.min(Math.max(frameSeconds, 0), this.maxFrame);
  let steps = 0;
  while (this.accumulator >= this.step && steps < this.maxSteps) {
    this.update(this.step);
    this.accumulator -= this.step;
    steps++;
  }
  if (steps === this.maxSteps && this.accumulator >= this.step) {
    const remainder = this.accumulator % this.step;
    this.dropped += this.accumulator - remainder;
    this.accumulator = remainder;
  }
  this.render(this.accumulator / this.step);
}
}

```

A fixed timestep does **not** automatically guarantee cross-platform determinism. Random-number control, floating-point behavior, update order, physics solver behavior, input ordering, workers, and external state can all matter.

6.4 Fixed timestep with interpolation

If the simulation ticks at one cadence and the display paints at another, interpolate selected **presentation transforms** between previous and current authoritative state.

```

function renderPosition(entity, alpha) {
  return {
    x: entity.previous.x + (entity.current.x - entity.previous.x) * alpha,
    y: entity.previous.y + (entity.current.y - entity.previous.y) * alpha
  };
}

```

Do not interpolate discrete state such as health, inventory, trigger flags, or scene identity. Teleports and respawns should normally synchronize previous/current transforms so the renderer does not invent motion across the world.

6.5 Background tabs and pause policy

requestAnimationFrame scheduling is browser-controlled and is commonly paused or throttled when content is not visible. Define what the product should do:

- freeze single-player simulation;
- reconcile wall-clock progress for an idle game;
- keep network authority separate from local render suspension;
- reset timing baselines on resume;

- never “catch up” an unbounded backlog by accident.

The toolkit preflight activates different validation obligations based on the declared timing model.

7. Browser Lifecycle

The browser can change conditions around the game without asking.

Relevant lifecycle surfaces include:

- visibilitychange;
- window focus/blur;
- resize and display migration;
- DPR changes;
- controller disconnect/reconnect;
- AudioContext state changes;
- WebGL context loss/restoration;
- storage quota/version upgrades;
- worker termination;
- page navigation/disposal.

Each subsystem needs an owner and a policy. A project that never uses WebGL does not need WebGL context-loss logic. A project that uses WebGL and owns raw GPU resources does.

7.1 Lifecycle invariant

Every acquired long-lived resource should answer four questions:

1. Who creates it?
2. Who owns it?
3. When does it become invalid?
4. Who releases or reconstructs it?

This applies to event listeners, timers, workers, object URLs, AudioNodes/contexts, DOM observers, WebGL textures/buffers/programs/framebuffers, caches, and asynchronous requests.

8. High-DPI and Coordinate Architecture

High-DPI scaling is **conditional on raster-backed rendering**, not a universal browser-game requirement.

Keep these coordinate spaces separate:

- CSS pixels;
- Canvas backing-store pixels;
- device pixel ratio (DPR);

- optional logical viewport units;
- world coordinates;
- camera/view coordinates.

A fixed logical viewport is one valid strategy, not the strategy.

```
function configureCanvas(canvas, logicalWidth, logicalHeight, maxDpr = 2) {
  const ctx = canvas.getContext('2d');
  if (!ctx) throw new Error('Canvas 2D unavailable');
  const dpr = Math.min(window.devicePixelRatio || 1, maxDpr);
  const fit = Math.min(innerWidth / logicalWidth, innerHeight / logicalHeight);
  const cssWidth = Math.max(1, Math.round(logicalWidth * fit));
  const cssHeight = Math.max(1, Math.round(logicalHeight * fit));
  canvas.style.width = `${cssWidth}px`;
  canvas.style.height = `${cssHeight}px`;
  canvas.width = Math.max(1, Math.round(cssWidth * dpr));
  canvas.height = Math.max(1, Math.round(cssHeight * dpr));
  ctx.setTransform(canvas.width / logicalWidth, 0, 0, canvas.height / logicalHeight, 0, 0);
  return { ctx, dpr, fit };
}
```

Use setTransform after resize rather than repeatedly multiplying an existing transform.

Pointer coordinates arrive in CSS/client space. Map them through the rendered canvas rectangle and then through camera/world transforms. Guard zero-sized/hidden surfaces before division.

DPR can change while a page remains open, including display migration and zoom. Re-read it when the rendering surface is reconfigured. An optional DPR cap is a performance/quality policy, not a correctness constant.

9. State Ownership and Scene Lifecycle

State separation is not a ritual requiring every project to invent an “authoritative simulation.” Apply the model when those categories exist.

Useful classes:

- **Authoritative gameplay state:** values used to decide gameplay truth—position, health, inventory, collision state, current turn.
- **Simulation history:** previous/current transforms retained for interpolation or replay.
- **Derived render state:** camera-space transforms, interpolated positions, batched GPU instance data.
- **Presentation-effects state:** shake envelopes, particles, flashes, UI transitions. Non-authoritative does not mean “mutate in render()”; give time-dependent effects their own update owner.
- **UI state:** focus, panel selection, menus, tooltips, settings.

- **Asset state:** requested/loading/ready/failed and current manifest/build identity.
- **Persistence state:** serialized schema version and storage transaction status.

A draw function should be observational whenever practical. If calling `render()` twice changes health, particle lifetime, collision, or scene state, display refresh can alter behavior.

9.1 Scene lifecycle

A useful scene/system lifecycle is:

```
initialize → enter → update → render → pause/resume → exit → dispose
```

Not every project needs explicit Scene classes. The invariant is ownership, not the class name.

Common leak pattern:

```
class Level {
  enter() {
    this.onKey = e => this.handleKey(e);
    window.addEventListener('keydown', this.onKey);
  }
  exit() {
    window.removeEventListener('keydown', this.onKey);
  }
}
```

The architecture linter emits LIFE101 when it sees several listener additions in a file without corresponding removals. That is a review signal, not proof of a leak—framework teardown may own the lifecycle elsewhere.

10. Input Architecture and Analog Deadzones

A formal action layer is **conditional**. Direct event handling can be completely reasonable for one-button interactions. Action mapping earns its complexity when the game needs multiple devices, rebinding, accessibility, buffered actions, tests, or device-independent gameplay logic.

Distinguish:

- **held/continuous** input—movement, aiming;
- **edge-triggered** input—jump pressed this tick;
- **released** input;
- **buffered** input—press accepted within a short gameplay window;
- **pointer/touch coordinates**—must be transformed into game space;
- **analog vectors**—need deadzone and magnitude policy.

10.1 Scaled radial deadzone

Axial deadzones create square response regions. A radial model preserves direction.

For raw vector ((x,y)), raw magnitude (m=), and deadzone (d):

- if (m d), output zero;
- direction uses the raw, unclamped magnitude;
- output magnitude maps the range ([d,1]) to ([0,1]), clamped at one.

```
function applyRadialDeadzone(x, y, deadzone = 0.15) {
  if (!Number.isFinite(x) || !Number.isFinite(y)) return { x: 0, y: 0 };
  if (!(deadzone >= 0 && deadzone < 1)) throw new RangeError('deadzone');
  const rawMagnitude = Math.hypot(x, y);
  if (rawMagnitude <= deadzone || rawMagnitude === 0) return { x: 0, y: 0 };
  const outputMagnitude = Math.min(1, (rawMagnitude - deadzone) / (1 - deadzone));
  return {
    x: (x / rawMagnitude) * outputMagnitude,
    y: (y / rawMagnitude) * outputMagnitude
  };
}
```

Test center, exact threshold, just outside threshold, diagonals, values beyond unit radius, malformed axes, and disconnect/reconnect.

PART III — ASSETS, RENDERING, ANIMATION, AND PHYSICS

11. Asset Authority and the Three-Stage Verified Lifecycle

The strongest rule in this toolkit is simple:

Generated outputs must become facts before runtime code depends on them.

An AI agent may specify an asset. That does not mean the asset exists. It must not invent:

- filenames;
- atlas coordinates;
- dimensions;
- frame counts;
- pivots;
- checksums;
- generated IDs;
- shader paths;
- audio paths.

11.1 The lifecycle

Stage 1 — Specification

Declare what is needed and any constraints.

Stage 2 — Generation/build

Create assets, process them, pack atlases if useful, and emit the machine-readable manifest.

Stage 3 — Verified integration

Validate the manifest and physical outputs; only then write runtime code against the emitted logical IDs.

This is a three-stage lifecycle. Do not call it “two-pass” while listing three passes. The linter cannot protect you from arithmetic shame.

11.2 Manifest structure

The toolkit ships `schemas/asset-manifest.schema.json` and a dependency-free validator. Example:

```
{
  "schemaVersion": 1,
  "atlases": {
    "characters": {
      "image": "characters.webp",
      "width": 2048,
      "height": 2048,
      "sha256": "optional-lowercase-hex"
    }
  },
  "assets": {
    "player.idle": {
      "kind": "atlasFrame",
      "atlas": "characters",
      "frame": {"x": 0, "y": 0, "w": 128, "h": 128}
    },
    "music.title": {
      "kind": "audio",
      "path": "audio/title.ogg"
    }
  }
}
```

Validate it:

```
python "$TOOLKIT/tools/manifest_validator.py" assets/manifest.json \
--project-root . \
--verify-files \
--verify-hashes
```

The validator checks schema version, supported kinds, frame bounds, path containment, file existence when requested, and checksums when present.

11.3 Source versus derived assets

Keep authority distinct:

- source artwork or generated source image;
- processed/trimmed derivative;
- atlas/build output;
- manifest metadata;
- runtime decoded object/GPU texture;
- cache.

The runtime manifest describes current build facts. It does not replace the source asset or provenance ledger.

11.4 Alpha and generated art

Prefer native alpha output when reliable. Other legitimate strategies include segmentation masks, chroma/background removal, color matting, manual cleanup, or intentionally retaining a background.

Thresholding a white background is a fallback, not a universal pipeline. It can create fringe artifacts and destroy pale interior pixels. Premultiplied/straight-alpha conventions must remain consistent through preprocessing and rendering.

11.5 Sprite sheets are legitimate

Conventional authored or tool-generated sprite sheets are excellent production assets. The anti-pattern is assuming that a generative image model will produce a mechanically exact multi-frame animation grid with stable anatomy, identical camera, and reliable frame geometry merely because the prompt asked nicely.

12. Rendering Path Selection

12.1 DOM/CSS

Use when semantics, forms, text, focus, document flow, responsive layout, and native accessibility are dominant.

Good fits:

- card/board games;
- text adventures;
- management interfaces;
- turn-based puzzles;

- menu-heavy experiences.

Costs appear when enormous numbers of independently animated elements force style/layout/paint work.

12.2 Canvas 2D

Use for immediate-mode 2D drawing, sprites, procedural graphics, moderate particle fields, custom composition, and small-to-medium action games.

Engineering concerns:

- DPR/backing-store policy;
- transform state;
- image smoothing;
- draw order and compositing;
- text metrics;
- clipping;
- pointer mapping;
- asset decode lifecycle.

Canvas does not need to be “graduated away from” when it meets the project requirements.

12.3 WebGL2

Use or retain WebGL2 when:

- the existing project already owns it successfully;
- required visuals need programmable shaders;
- GPU particles/lighting/postprocessing are genuine requirements;
- profiling identifies a Canvas limitation for the representative workload.

Standard WebGL2 instanced APIs include `drawArraysInstanced`, `drawElementsInstanced`, and `vertexAttribDivisor`. Desktop OpenGL names are not browser APIs merely because they resemble them.

Do not make “over 1,000 sprites” a migration rule. A few huge translucent effects can be more expensive than thousands of tiny opaque quads.

12.4 Existing frameworks

If the project already uses Phaser, PixiJS, another 2D renderer, or a coherent custom renderer, preserve it unless the change actually requires migration.

12.5 Hybrid interfaces

A useful common architecture is Canvas/WebGL for the playfield and semantic DOM for menus, settings, inventories, text, and accessibility surfaces. The render path is not required to be ideologically pure.

13. WebGL2 Resource and Failure Model

Raw WebGL code must treat resource creation and context lifecycle as explicit state.

13.1 Context creation

```
const gl = canvas.getContext('webgl2', {
  alpha: true,
  antialias: false
});
if (!gl) throw new Error('WebGL2 unavailable');
```

powerPreference is a hint, not a command to the browser, and should not be hard-coded to high-performance without a project reason.

13.2 Shader failure is not a black screen

Compile/link failures should surface logs immediately. Development builds should fail loudly with the shader stage and log.

13.3 Context loss

If the project owns raw WebGL resources, handle `webglcontextlost` and `webglcontextrestored`. After restoration, reconstruct textures, buffers, programs, VAOs, framebuffers, and render state from reloadable/CPU-side sources. Old GPU handles are not current truth.

13.4 Batching and instancing

Instancing solves repeated CPU draw submission for compatible geometry/material state. It does not automatically solve fill-rate, shader cost, blending, overdraw, texture bandwidth, or sorting.

Measure total frame cost rather than worshipping the draw-call counter.

14. Lighting, Normal Maps, and Postprocessing

These are **optional visual capabilities**.

14.1 Luminance-derived normals

A Sobel or similar filter over diffuse luminance infers brightness gradients, not geometry. Painted highlights, shadows, outlines, emissive areas, and material-color changes can become false bumps or trenches.

Treat luminance-derived normals as an artistic approximation. Validate orientation with controlled lights from multiple directions.

14.2 Postprocessing

Bloom, blur, chromatic offsets, distortion, and color grading typically require offscreen framebuffers and full-screen passes. They cost memory bandwidth and fragment work, especially at high DPR.

Provide a reduced-effects path when effects involve flashing, shake, rapid distortion, or other sensory load. A lowered-resolution intermediate target can be a valid performance tradeoff when visual quality permits.

Explicitly define color-space and alpha conventions between passes; compiled shaders can still produce incorrect halos and bloom when the pipeline mixes straight/premultiplied alpha or nonlinear/linear-light assumptions.

15. Animation and Camera

Choose animation technique from the art and movement requirements.

- **Frame animation:** excellent for authored pixel art, hand-drawn clips, prerendered effects.
- **Sprite sheets/atlasses:** efficient packaging; frame metadata belongs in build data.
- **Part-based transforms:** useful for articulated generated assets, weapons, turrets.
- **Tweens:** UI transitions, recoil, pickups, scripted motion.
- **Procedural animation:** useful where motion responds to live state.
- **IK:** only when target-reaching limbs/terrain placement earn the solver complexity.
- **Shader animation:** UV scrolls, dissolves, water, non-authoritative effects.

15.1 Camera state

Separate:

- camera target;
- logical/simulated camera state;
- optional interpolated render camera;
- presentation-only offsets such as shake.

Frame-rate-independent exponential smoothing can be expressed as:

```
function smoothToward(current, target, lambda, dt) {
  return current + (target - current) * (1 - Math.exp(-lambda * dt));
}
```

This is exponential smoothing. Do not call it a “critically damped spring” unless you actually implement the spring/velocity model.

Layered sine/cosine shake is legitimate, but it is not Perlin noise. Names should describe algorithms that exist, not ones that sound expensive.

16. Collision and Physics Escalation

Start with the cheapest shape that expresses gameplay correctly:

1. point/point or point/shape;
2. circle/radius;
3. AABB;
4. capsule/segment;
5. swept/continuous tests for fast objects;
6. spatial broadphase when candidate work is measured as a problem;
7. SAT/convex polygon narrowphase when rotated/convex precision is actually needed;
8. pixel/alpha masks for truly silhouette-dependent mechanics;
9. a dedicated physics engine when constraints, stacks, joints, rigid-body solving, or continuous collision exceed the value of custom maintenance.

16.1 Spatial partitioning

A spatial hash/grid can reduce candidate work, but practical performance depends on cell size, distribution, clustering, object extent, duplicate suppression, update frequency, and narrowphase cost. Never describe it as universally $O(N)$ in a way that hides pathological clustering.

Benchmark at least a typical scene and a clustered/worst representative scene. Record candidate counts, not just frame rate.

PART IV — PERFORMANCE, CONCURRENCY, AUDIO, STORAGE, AND SECURITY

17. Memory and Performance Methodology

JavaScript is garbage collected. “Zero-GC architecture” is marketing unless narrowly scoped and actually measured. The useful engineering target is **allocation awareness in hot paths**.

17.1 Profile before changing data style

Establish:

- CPU update time;
- rendering time;
- DOM style/layout/paint if relevant;
- GPU/fragment pressure where measurable;
- overdraw/blending;
- asset decode/upload spikes;

- allocation rate and GC events;
- collision candidate/narrowphase cost;
- worker messaging;
- input-to-presentation latency;
- retained memory over repeated scene transitions.

Only then choose interventions.

17.2 Possible interventions

- reusable scratch objects;
- typed arrays for dense numeric/GPU data;
- object pools for proven high-churn entity types;
- culling;
- batching;
- reduced render scale/DPR cap;
- fewer full-screen passes;
- smaller candidate sets;
- asset decode/preload policy;
- worker offload for independent heavy computation.

Pooling has costs: reset bugs, stale state, use-after-release, peak-sized pools, and readability damage. Keep it only when it solves a measured problem.

17.3 Benchmark statement form

A valid performance claim names the scenario:

In scenario X, on device Y, browser Z, resolution/DPR R, build B produced percentile P under workload W.

“One draw call,” “10,000 sprites,” or “60 FPS” without workload/environment is not a portable guarantee.

18. Workers, OffscreenCanvas, and Shared Memory

Workers are **conditional**. Move work off the main thread only when profiling identifies a suitable independent workload and the messaging/ownership boundary is clean.

Good candidates can include:

- pathfinding;
- terrain generation;
- expensive turn resolution;

- offline processing;
- some simulation workloads.

Costs include serialization/structured clone, transfer ownership, synchronization, race conditions, duplicate state, build complexity, and harder debugging.

18.1 Transferables first

Transferable ArrayBuffer objects can avoid copies in suitable pipelines. SharedArrayBuffer is not the default “fast path.” Use it only when shared-memory semantics materially solve a measured problem.

18.2 SharedArrayBuffer security gate

Modern browser shared-memory use is tied to cross-origin isolation. The project contract requires workers: true plus deployment headers for SharedArrayBuffer activation:

```
{
  "deploymentHeaders": {
    "Cross-Origin-Opener-Policy": "same-origin",
    "Cross-Origin-Embedder-Policy": "require-corp"
  }
}
```

The linter emits errors when the declared SharedArrayBuffer contract lacks compatible COOP/COEP settings. Runtime validation must also confirm `window.crossOriginIsolated === true` and that isolation has not broken required third-party resources.

19. Web Audio

Web Audio is conditional. Use it for dynamic routing, buses, synthesis, analysis, spatialization, or programmatic control that media elements do not provide sufficiently.

Browser autoplay policy may prevent an AudioContext from reaching running until appropriate user activation. Do not assume every context always starts in exactly the same state; query it and handle resume failure.

```
async function ensureAudioRunning(ctx) {
  if (ctx.state === 'closed') throw new Error('AudioContext closed');
  if (ctx.state !== 'running') await ctx.resume();
  if (ctx.state !== 'running') throw new Error(`AudioContext state: ${ctx.state}`);
}
```

A useful bus graph:

```
SFX  -> SFX gain ─┐
Music-> Music gain ┤─> Master gain -> destination
UI   -> UI gain  ─┘
```


Do not describe dynamically created Web Audio nodes as allocation-free. If sound creation correlates with stutter, profile it.

20. Persistence and Schema Evolution

Persistence is conditional. Use the simplest store that fits the data and failure model.

- localStorage: small synchronous configuration/simple snapshots.
- IndexedDB: larger structured/asynchronous data, richer save systems, caches.

Persist a schema version. Define migration paths from supported older versions. Handle malformed/corrupt data and quota/storage errors. With IndexedDB, account for blocked upgrades and other open tabs/connections.

A “Saved” indicator should mean the transaction committed successfully, not that a save function was called.

Browser-local storage is user/device/browser state and can be cleared. It is not a substitute for account/server authority where durable cross-device persistence is required.

21. Accessibility and Browser UI

Accessibility is part of architecture when interaction depends on it.

Prefer semantic DOM for menus/settings/forms where practical, even when gameplay is Canvas/WebGL. Address:

- keyboard operability;
- visible focus;
- remapping when controls are complex;
- touch targets and no hover-only interactions;
- reduced motion and sensory effects;
- readable text/contrast;
- subtitles/captions where applicable;
- focus traps and modal behavior;
- browser zoom unless gameplay has a narrowly justified exception.

prefers-reduced-motion is a useful default signal. If the game exposes an explicit setting, that explicit user choice can take precedence over the inherited default.

22. Security and Deployment Boundaries

The browser client is not a secret store.

FORBIDDEN:

- private keys in frontend source;

- long-lived API/service credentials in client code;
- bearer tokens committed into source or generated reports;
- secret material in normal logs/screenshots/analytics;
- privileged setuid helpers as a default deployment mechanism;
- hidden publish/deploy side effects inside ordinary build commands.

Separate:

BUILD → PACKAGE → UPLOAD → DEPLOY → PUBLISH

Authorization should become stronger as the workflow crosses those boundaries.

22.1 Signed URLs

A short-lived signed upload URL can be a reasonable temporary capability issued by a trusted backend. It is still a bearer capability: anyone possessing an active URL may be able to perform the signed operation until expiration/invalidity.

Therefore:

- use short expiry;
- scope method/object path;
- constrain headers/content policy where the provider supports it;
- never paste signed URLs into routine logs, screenshots, issue trackers, or analytics;
- keep signing authority server-side and least-privileged;
- validate uploaded type/size/content at a trusted layer when needed.

The linter flags ordinary credential-like query parameters; a reviewed signed-URL implementation may need a narrow suppression or an isolated non-source runtime surface. Suppression is not proof—record meaningful exceptions in an ADR.

PART V — TESTING, EVIDENCE, AND THE TOOLKIT

23. Testing and Evidence Ladder

Use the weakest truthful evidence label, not the strongest flattering one.

1. source inspected;
2. syntax/type/schema verified;
3. unit/fixture test executed;
4. integration path executed;
5. browser path executed;
6. visual/player journey verified;

7. representative performance workload measured.

Different claims require different evidence. A shader source parsing test does not prove it compiles on the target GPU. A page loading without console errors does not prove the player can complete the requested interaction. A screenshot does not prove persistence or cleanup.

23.1 Conditional verification

Validate only enabled capabilities, but validate them decisively.

Capability	Minimum proof examples
DOM	keyboard/focus path, semantic control inspection
Canvas	resize/DPR/pointer mapping, browser smoke
fixed timestep	max-step/backlog fixture, integration stability
interpolation	previous/current blend and teleport/snap fixture
generated assets	manifest schema, physical files, bounds/checksums
gamepad	center/edge/diagonal/disconnect
Web Audio	user-activation startup, mute/resume/failure
WebGL2	context creation, shader compile/link, context loss if owned
workers	messaging, termination, before/after responsiveness
SharedArrayBuffer	crossOriginIsolated, headers, synchronization/race fixtures
persistence	round trip, migration, corruption/quota behavior
deployment	no exposed secrets, authorization boundary, dry-run/staging proof

24. Toolkit Components

The companion toolkit turns the handbook into an executable workflow.

24.1 bootstrap_project.py

Creates an initial contract and architecture-document templates without overwriting an existing contract.

```
python "$TOOLKIT/tools/bootstrap_project.py" . --name "My Game"
```

Outputs:

- architecture.project.json;

- docs/architecture/ADR.md;
- docs/architecture/AI_TASK_PACKET.md;
- docs/architecture/VALIDATION_REPORT.md.

24.2 architecture_preflight.py

Compiles the project contract into **activated handbook areas** and mandatory proof obligations. It also lists deactivated capabilities so an agent has explicit “do not add this yet” boundaries.

```
python "$TOOLKIT/tools/architecture_preflight.py" architecture.project.json \
--project-root . \
--out ACTIVATION.md
```

24.3 architecture_linter.py

Static, contract-aware linter. It deliberately does not pretend to semantically understand the whole game.

```
python "$TOOLKIT/tools/architecture_linter.py" . \
--config architecture.project.json \
--json architecture-lint.json \
--strict
```

It detects high-confidence problems such as secret-like literals, desktop OpenGL names in browser source, missing generated-asset manifests, SharedArrayBuffer isolation mismatches, direct asset paths under a manifest-driven contract, suspicious listener ownership, missing Web Audio resume handling, and contract/code drift.

Warnings are review obligations, not convictions.

24.4 manifest_validator.py

Validates generated/runtime asset facts.

```
python "$TOOLKIT/tools/manifest_validator.py" assets/manifest.json \
--project-root . --verify-files --verify-hashes
```

It checks frame bounds, supported kinds, path containment, physical existence, and optional checksums.

24.5 checklist_generator.py

Produces a validation checklist from the declared capabilities so irrelevant subsystems do not become mandatory ceremony.

```
python "$TOOLKIT/tools/checklist_generator.py" architecture.project.json \
--out VALIDATION_CHECKLIST.md
```

24.6 browser_smoke.mjs

Optional Playwright harness for a running local/test application.

```
node "$TOOLKIT/tools/browser_smoke.mjs" http://127.0.0.1:5173 \
--ready "() => document.querySelector('[data-app-ready]')
```

It captures page errors, console errors, failed requests, navigation failure, optional readiness expression, and optional screenshot. If Playwright is not installed, it exits as a clear **SKIP**, not a fake pass.

A smoke test is not visual QA and is not a gameplay-completion test.

24.7 run_project_checks.py

This is the normal entry point.

```
python "$TOOLKIT/tools/run_project_checks.py" . --strict-lint
```

Optional flags:

```
--execute-project-commands  run the explicit validation.commands in the contract
--browser                   run optional Playwright browser smoke
--verify-asset-files        validate declared generated-asset files
--verify-asset-hashes       also validate declared SHA-256 values
--strict-lint               make warnings fail the linter stage
```

It writes .architecture-checks/ACTIVATION.md, VALIDATION_CHECKLIST.md, architecture-lint.json, optional manifest-validation.json, and SUMMARY.json.

Project commands are **not executed by default**. That boundary is intentional; reading a repository should not silently execute arbitrary shell commands.

24.8 toolkit_selftest.py

Validates the toolkit itself:

```
python "$TOOLKIT/tools/toolkit_selftest.py"
```

It compiles Python tools, checks the JavaScript smoke harness syntax when Node is present, exercises clean and intentionally failing linter/manifest fixtures, and validates all ten shipped example contracts.

24.9 Schemas

- schemas/architecture-project.schema.json
- schemas/asset-manifest.schema.json

These make the contract formats inspectable by editors, CI, and other tools.

24.10 CI template

templates/github-actions-architecture.yml shows how to run the linter/manifest checks in GitHub Actions. Adapt the toolkit path/install method to the repository. Add browser smoke only when the CI job actually starts the app and installs Playwright.

25. Architecture Linter Rule Set

The exact current catalog is shipped in tools/LINTER_RULES.md. Core rules include:

Code	Class	Trigger
SEC001	error	private-key material
SEC002	error	AWS access-key-like literal
SEC003	error	sk-... API-key-like literal
SEC004	error	JWT-like literal
SEC005	error	credential-like query parameter in URL
SEC101/102	error	SharedArrayBuffer contract missing isolation headers
SEC103	warning	SharedArrayBuffer code exists but contract does not activate it
WEBGL101	error	desktop OpenGL-style API name in browser source
CFG101	error	generated assets active without manifest path
ASSET101	error	declared manifest missing
ASSET102	warning	direct asset literal outside approved catalog/loader surface
LIFE101	warning	several listener additions with no visible removals
TIME101	warning	fixed timing declared without validation commands
TIME102	info	several RAF calls; review loop ownership
AUDIO101/102	warning	Web Audio contract/source lifecycle mismatch
DEP101	warning	deployment automation without declared validation

25.1 Suppression

The project contract supports:

```
"lint": {
  "ignoreCodes": [],
  "ignorePaths": []
}
```

Suppress only after review. Use path suppression for generated/vendor/fixture text that should not participate. Use code suppression for a deliberate exception. If a suppression affects security, deployment, generated-asset authority, or a foundational architectural rule, capture the rationale in an ADR.

25.2 What the linter deliberately cannot prove

It cannot prove:

- that a listener leaks;
- that a renderer is performant;
- that fixed timestep is necessary;
- that a secret-like string is a real active credential;
- that a game is accessible;
- that a generated asset is visually correct;
- that a browser journey works;
- that a WebGL shader behaves correctly on target hardware.

Those require appropriate human/runtime evidence. The linter narrows attention; it does not replace engineering judgment.

26. Architecture Decision Records and Agent Handoffs

Use an ADR when the decision materially changes cost, lifecycle, deployment, dependency, or reversibility.

The included template asks for:

- problem;
- evidence;
- options;
- decision;
- rejected alternatives;
- cost;
- validation;
- rollback.

The included AI task packet gives coding agents a bounded handoff with current architecture, protected behavior, exact change scope, validation, and prohibited collateral changes.

The rule is not “write more paperwork.” The rule is “do not let a consequential decision disappear into chat.”

27. AI Coding Agent Operating Contract

The following can be placed in project instructions or included in an implementation packet:

SKILL: Requirement-Driven 2D Browser Game Engineering

1. Inspect before changing. Read the current renderer/framework, lifecycle owner, asset loader/catalog, relevant state owner, tests, and build surfaces first.
2. No unsolicited engine rewrites. Preserve functional project architecture unless the requested outcome requires migration and evidence supports it.
3. Architecture follows the project contract. Do not activate a capability merely because the handbook describes it.
4. Generated outputs are not facts until they exist. Never invent generated file names, atlas frames, dimensions, pivots, IDs, or checksums.
5. Own state explicitly. Drawing/presentation must not secretly mutate authoritative gameplay state. Time-dependent effects need an update owner.
6. Handle failure visibly. Do not swallow missing assets, malformed manifests, shader failures, AudioContext failures, storage errors, or worker failures.
7. Measure before performance redesign. Do not introduce pooling, spatial broadphase, workers, WebGL migration, instancing, or resolution changes from superstition.
8. Respect lifecycle. Pair acquired listeners, workers, GPU resources, object URLs, audio graphs, observers, and async tasks with ownership and release/invalidity rules.
9. Respect browser security. Never expose secrets or persistent bearer credentials in client source, generated assets, ordinary logs, screenshots, or analytics.
10. Run contract tooling before completion: preflight/linter, manifest validation when activated, declared project tests/builds when authorized, and the real user path.
11. Use truthful evidence labels. Source presence, syntax, test execution, browser execution, visual verification, and performance measurements are distinct.
12. Stop when requirements pass. Do not continue with unsolicited dependencies, refactors, engine rewrites, or decorative optimization.

PART VI — TEN CONCRETE BUILD EXAMPLES

The toolkit ships exactly ten example project contracts under `examples/`. They are not toy “look what Canvas can do” snippets. Each demonstrates a different architecture-selection decision and shows how to use the handbook/tooling to turn a concept into a bounded build.

28. Example One — Accessible DOM Card Battler

Make: a turn-based card battler playable with mouse or keyboard, with saved settings and semantic controls.

Contract: `renderPath=dom, timingModel=event, persistence=localStorage`.

Why this architecture: turns happen because a player chooses an action; there is no reason to maintain a permanent render loop or canvas scene. Native buttons and DOM structure make focus, text, keyboard, and screen-reader behavior easier to own.

Build sequence:

1. Bootstrap the project contract.
2. Represent cards and actions as semantic DOM elements.
3. Model match state as explicit turn transitions.
4. Store only small settings/deck snapshots in a versioned `localStorage` envelope.
5. Run preflight/linter/checklist.
6. Complete a keyboard-only match as the decisive player-path test.

Do not add: Canvas, fixed timestep, physics, asset atlas, WebGL, worker.

Useful acceptance evidence: focus order remains visible and logical; the match is completable without pointer input; reload restores supported settings; malformed old storage falls back or migrates explicitly.

Starter contract: `examples/01-dom-card-battler/architecture.project.json`.

29. Example Two — High-DPI Canvas Arcade Dodger

Make: a one-screen dodge game with smooth keyboard movement, resize-safe Canvas rendering, and simple sound effects.

Contract: `renderPath=canvas2d, timingModel=variable, webAudio=true`.

Why this architecture: movement is simple enough for bounded variable delta. Canvas provides an uncomplicated custom playfield. Web Audio is activated because sound is part of the experience, not because every game needs a bus graph.

Build sequence:

1. Implement one Canvas with explicit DPR/resize handling.
2. Use a bounded variable-delta loop.

3. Clear held input on blur/hidden transitions.
4. Initialize/resume audio from the Start control.
5. Add circle/AABB collision only as required.
6. Verify pointer/geometry alignment after resize and different DPR values.

Do not add: fixed interpolation, WebGL, broadphase, generated asset manifests unless the game later acquires those requirements.

Starter contract: `examples/02-canvas-arcade-dodger/architecture.project.json`.

30. Example Three — Fixed-Step Platformer

Make: a small platformer whose collision simulation remains stable while motion renders smoothly on high-refresh displays.

Contract: `renderPath=canvas2d, timingModel=fixed_interpolated, gamepad=true, webAudio=true`.

Why this architecture: collision/integration benefits from a stable step. Interpolation is activated to prevent visible stepping. Input abstraction earns its cost because keyboard and gamepad share gameplay actions.

Build sequence:

1. Implement fixed simulation with a maximum step count and observable backlog policy.
2. Retain previous/current transform state for drawable positions.
3. Snap history on teleport/respawn.
4. Add keyboard/gamepad actions and radial deadzone fixtures.
5. Start with AABB collision.
6. Profile before considering broadphase.

Decisive tests: injected long frame cannot run unbounded ticks; simulation speed remains consistent under varied render cadence; respawn does not interpolate across the map; gamepad disconnect returns neutral input.

Starter contract: `examples/03-fixed-platformer/architecture.project.json`.

31. Example Four — Generated-Art Top-Down Adventure

Make: a top-down exploration game whose AI-generated sprites are integrated without fabricated filenames, dimensions, or atlas coordinates.

Contract: `Canvas, fixed+interpolated timing, generated assets with assets/manifest.json, IndexedDB saves, Web Audio`.

Why this architecture: the interesting architectural problem is not rendering—it is **authority across a generative asset pipeline**. The manifest converts uncertain generation outputs into verified runtime facts.

Build sequence:

1. Write an asset specification for player, NPCs, environment, UI.
2. Generate/process assets and emit manifest.
3. Run `manifest_validator.py --verify-files --verify-hashes`.
4. Only then implement runtime lookups by logical ID.
5. Fail explicitly for unknown IDs or stale manifests.
6. Add versioned IndexedDB save/migration fixtures.

Agent boundary: a prompt requesting `player_walk.webp` does not make that filename real. Runtime code waits for the build manifest.

Starter contract: `examples/04-generated-art-adventure/architecture.project.json`.

32. Example Five — WebGL Particle Arena

Make: a twin-stick arena game with custom particle/shader effects and an optional reduced-effects mode.

Contract: `renderPath=webgl2`, fixed+interpolated timing, gamepad, Web Audio, postprocessing.

Why this architecture: WebGL is activated by the visual design—programmable effects/particles—not a magic entity count.

Build sequence:

1. Establish context/shader compile-link failure reporting.
2. Define GPU resource ownership and disposal.
3. Build the base sprite renderer before postprocessing.
4. Add compatible batching only around actual material/blend boundaries.
5. Add one postprocessing effect at a time and profile effect-on/effect-off.
6. Provide reduced-effects controls.
7. Exercise context loss/restoration if raw resources are project-owned.

Evidence: shader errors become actionable; representative frame traces identify CPU/GPU costs; effects can be reduced without destroying gameplay information.

Starter contract: `examples/05-webgl-particle-arena/architecture.project.json`.

33. Example Six — Accessible Puzzle Museum

Make: a room-based puzzle collection designed around semantic interaction, keyboard navigation, text, and reduced motion.

Contract: DOM, event-driven, localStorage.

Why this architecture: semantic browser UI is the product, not an overlay around a canvas. Accessibility is therefore structural rather than bolted on.

Build sequence:

1. Model puzzle state/event transitions.
2. Use headings, buttons, lists, dialogs, and status regions with appropriate semantics.
3. Maintain visible focus and modal focus behavior.
4. Read reduced-motion preference as a default and expose an explicit preference if needed.
5. Persist small completion/settings state with schema version.
6. Test every puzzle keyboard-only.

Do not add: a canvas just to make transitions more cinematic. CSS/Web Animations can remain optional presentation layers.

Starter contract: `examples/06-accessible-puzzle-museum/architecture.project.json`.

34. Example Seven — IK Creature Sandbox

Make: a 2D creature whose articulated limbs reach targets and plant on uneven terrain using optional two-bone IK.

Contract: Canvas variable-delta, generated assets/manifest, inverse kinematics enabled.

Why this architecture: IK is activated by an explicit motion requirement. Generated body parts require verified pivots/IDs, so asset authority matters as much as the solver.

Build sequence:

1. Generate or author body parts with stable pivot metadata.
2. Validate the manifest before constructing the rig.
3. Implement two-bone solver with reach/near-zero clamps.
4. Keep solved limb presentation out of authoritative gameplay unless collisions intentionally depend on it.
5. Test unreachable, folded, near-zero, and mirrored poses.

Failure to prevent: regeneration silently changes a body-part ID or pivot and makes the rig “mysteriously” explode. Manifest validation should convert that into an explicit integration failure.

Starter contract: `examples/07-ik-creature-sandbox/architecture.project.json`.

35. Example Eight — Swarm Collision Lab

Make: a benchmark/gameplay laboratory comparing naive collision pairing against a spatial broadphase under uniform and clustered distributions.

Contract: Canvas, fixed timestep, spatial broadphase activated.

Why this architecture: this is deliberately an evidence-producing project. It demonstrates how a capability should be activated: by measuring a real problem and comparing interventions under the same workload.

Build sequence:

1. Keep the naive baseline.
2. Implement the spatial grid as a separate selectable path.
3. Record broadphase time, candidate pairs, narrowphase calls, and frame percentiles.
4. Sweep several cell sizes.
5. Test uniform and clustered scenes.
6. Retain the broadphase only where the measured tradeoff is worthwhile.

Do not claim: universal $O(N)$ behavior or a universal entity-count threshold.

Starter contract: `examples/08-swarm-collision-lab/architecture.project.json`.

36. Example Nine — Offline Strategy Simulation

Make: an offline-first management/strategy game whose expensive turn-resolution logic moves to a Worker only after the main-thread workload is measured.

Contract: DOM, event timing, workers, IndexedDB.

Why this architecture: presentation is interface-heavy, so DOM is appropriate. Long turn calculations can become independent worker jobs. Persistence is structured and larger than simple settings.

Build sequence:

1. Implement turn resolution on main thread first and benchmark it.
2. Define worker message schemas and immutable job/result boundaries.
3. Move bounded pure computation to the worker.
4. Define cancellation/termination/restart behavior.
5. Store versioned saves in IndexedDB.
6. Test blocked upgrades, corruption, and quota errors.
7. Compare main-thread responsiveness before and after worker migration.

Evidence requirement: the worker remains only if it improves the actual interaction under a matched scenario.

Starter contract: `examples/09-offline-strategy-sim/architecture.project.json`.

37. Example Ten — Cross-Origin-Isolated Field Simulator

Make: an advanced particle/field simulator that intentionally uses shared memory between a Worker and WebGL renderer.

Contract: WebGL2, workers, SharedArrayBuffer, postprocessing, COOP/COEP declared.

Why this architecture: shared memory is activated only after transfer/copy overhead is demonstrated to matter. The security/deployment boundary is part of the architecture, not an afterthought.

Build sequence:

1. Build a transferable-buffer baseline.
2. Measure whether copy/ownership churn is actually material.
3. Activate SharedArrayBuffer only if evidence justifies it.
4. Configure COOP/COEP and verify crossOriginIsolated in the deployed test environment.
5. Define synchronization/race ownership explicitly.
6. Verify isolation does not silently break required third-party resources.
7. Keep a non-shared fallback or explicit unsupported state if product requirements need one.

Starter contract: `examples/10-shared-memory-field/architecture.project.json`.

PART VII — OPERATING THE TOOLKIT IN REAL WORK

38. Recommended Project Lifecycle

At project start

1. Bootstrap or write the contract.
2. Run preflight.
3. Review activated/deactivated capabilities.
4. Establish initial validation commands.
5. Commit the contract with the project.

Before a substantial feature

1. Read the contract and current repository architecture.
2. Determine whether the feature activates a new capability.
3. If yes, update contract and create an ADR when the change is consequential.
4. Give the coding agent a bounded task packet.

During implementation

1. Generated/build outputs become facts before runtime integration.
2. Keep errors visible.
3. Run the linter after structural changes.
4. Profile before performance-driven escalation.

Before completion

```
python "$TOOLKIT/tools/run_project_checks.py" . \
--strict-lint \
--verify-asset-files
```

Then run the actual project validation commands and browser/player journey appropriate to the request. Use `--execute-project-commands` only when you intentionally authorize commands stored in the contract. Use `--browser` when the test server is running and Playwright is available.

In CI

Run contract lint/manifest checks on pull requests. Browser smoke should be a separate job with an explicitly started server. Do not make CI “pass” by suppressing every warning; review warnings, fix the code/contract, or document a narrow exception.

39. Failure Interpretation

A toolkit failure should tell you **which layer is wrong**.

- **Contract validation fails:** project declaration is malformed or internally inconsistent.
- **Preflight seems wrong:** contract does not describe the actual project.
- **Linter error:** high-confidence source/contract violation; repair before completion.
- **Linter warning:** inspection required; may be a legitimate architecture owned elsewhere.
- **Manifest error:** generated/runtime facts are invalid; do not route around the manifest with hard-coded paths.
- **Browser smoke SKIP:** dependency/environment absent; this is not a pass or fail.
- **Browser smoke FAIL:** runtime page path produced concrete errors; repair before claiming that path works.
- **Project command failure:** project-specific tests/builds failed. The architecture toolkit does not overrule them.

40. What This Toolkit Does Not Do

It does not:

- choose game design for you;
- generate art by itself;
- prove performance statically;

- prove accessibility with regex;
- replace visual/player QA;
- replace framework-specific expertise;
- force a project to use Canvas or WebGL;
- automatically execute project shell commands without an explicit flag;
- upload or deploy builds;
- hide an implementation failure to preserve a green report.

Its purpose is narrower and more useful: **make architecture decisions explicit, keep generated facts factual, catch recurring high-confidence mistakes, and make completion evidence legible.**

PART VIII — CURRENT PLATFORM SOURCE MAP

The following platform claims were rechecked against current official/specification sources during this release:

- **WebGL2 core and instancing:** Khronos WebGL 2.0 specification and MDN WebGL2 API references.
- **WebGL context loss/restoration and context creation errors:** MDN/API surfaces linked to WebGL specifications.
- **requestAnimationFrame/visibility behavior:** MDN and HTML/Page Visibility specification surfaces.
- **DPR changes:** MDN window.devicePixelRatio guidance.
- **OffscreenCanvas:** WHATWG HTML canvas/offscreen surfaces and MDN transfer documentation.
- **SharedArrayBuffer/cross-origin isolation:** MDN COOP/COEP/crossOriginIsolated guidance and HTML security model.
- **Web Audio:** Web Audio specification plus current browser best-practice documentation.
- **IndexedDB/storage quota:** current MDN IndexedDB and Storage API documentation.
- **Signed upload URLs:** Google Cloud Storage signed-URL documentation; active URLs are temporary bearer capabilities and must be handled accordingly.

Primary reference URLs are recorded in the release README and this package's source-verification notes. Platform support matrices must still be checked against the target browsers at implementation time; a source-verified API fact is not proof that every target browser/device meets a project requirement.

41. Glossary

- **Architecture contract:** machine-readable project capability declaration.
- **Activation evidence:** requirement or measurement that justifies a capability.
- **Authoritative state:** state used to decide gameplay truth.
- **Backlog policy:** what fixed-step simulation does when accumulated time exceeds safe catch-up work.
- **Broadphase:** coarse collision stage that proposes candidate pairs.
- **Narrowphase:** accurate collision/contact test on candidates.
- **DPR:** device pixel ratio used in CSS/backing-store relationships.
- **Derived render state:** disposable presentation data computed from authoritative/history state.
- **Manifest:** machine-readable authority mapping logical assets to actual build outputs/metadata.
- **Hot path:** frequently executed path where measured cost can affect responsiveness.
- **Bearer capability:** possession itself grants the represented temporary authority.
- **Cross-origin isolation:** browser security state required by restricted shared-memory capabilities.
- **ADR:** Architecture Decision Record.

42. Final Operational Checklist

Before calling a project change complete:

- ☐ The architecture contract describes the current implementation.
- ☐ No new capability was introduced without requirement/evidence.
- ☐ Existing protected player journeys remain intact.
- ☐ Generated asset references come from current verified manifests.
- ☐ Architecture linter has no unresolved errors.
- ☐ Warnings are repaired or deliberately reviewed/suppressed.
- ☐ Enabled subsystem validation checklist is complete.
- ☐ Project-native tests/builds pass where required.
- ☐ Browser/player-visible behavior is actually exercised where required.
- ☐ Performance claims are backed by representative measurement.
- ☐ Secrets/bearer capabilities are absent from persistent client/log surfaces.
- ☐ Resource owners have disposal/recovery behavior where lifecycle requires it.
- ☐ Completion reporting distinguishes source, syntax, test, browser, visual, and performance evidence.

- ☐ The work stops when the requested outcome is proven.
-

Final Rule

Do not build the most advanced architecture you can describe. Build the least complicated architecture you can prove is sufficient—and leave behind enough contract, tooling, and evidence that the next human or agent does not have to rediscover why.